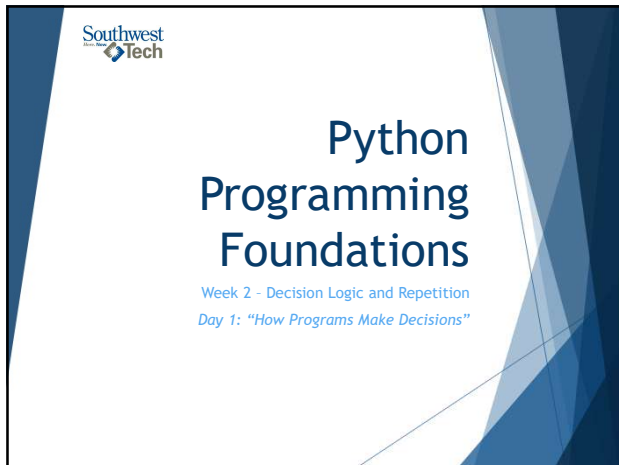
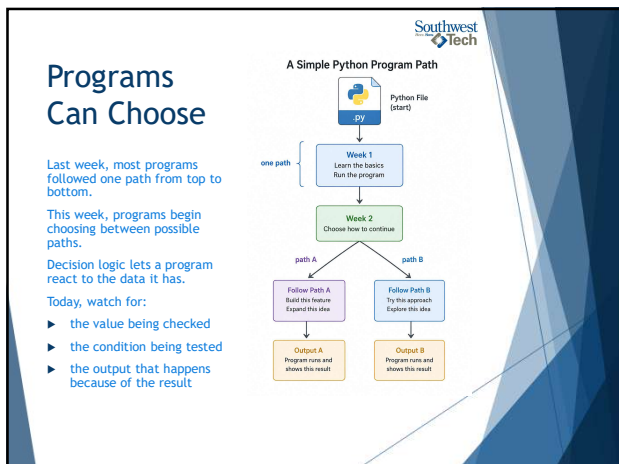
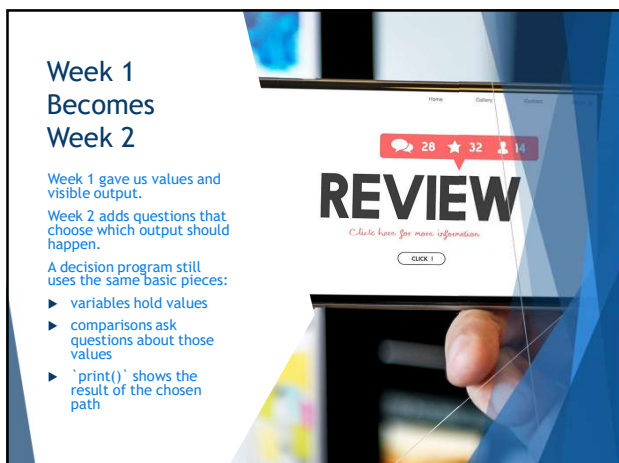




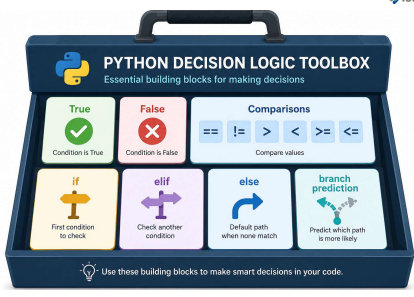
1



2



3



PYTHON DECISION LOGIC TOOLBOX
Essential building blocks for making decisions

True
Condition is True

False
Condition is False

Comparisons
Compare values

if
First condition to check

elif
Check another condition

else
Default path when none match

branch prediction
Predict which path is more likely

Use these building blocks to make smart decisions in your code.

Today's Toolbox

7

What We Will "Skip" For Now

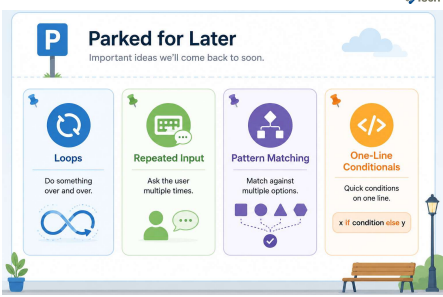
We will skip loops, repeated input, pattern matching, and compact one-line conditionals for now.

Today's job is to understand one decision path clearly.
That means the program should be small enough that you can trace:

- ▶ the starting value
- ▶ the condition
- ▶ the branch
- ▶ the final output



8



Parked for Later
Important ideas we'll come back to soon.

Loops
Do something over and over.

Repeated Input
Ask the user multiple times.

Pattern Matching
Match against multiple options.

One-Line Conditionals
Quick conditions on one line.
x if condition else y

Parked Concepts

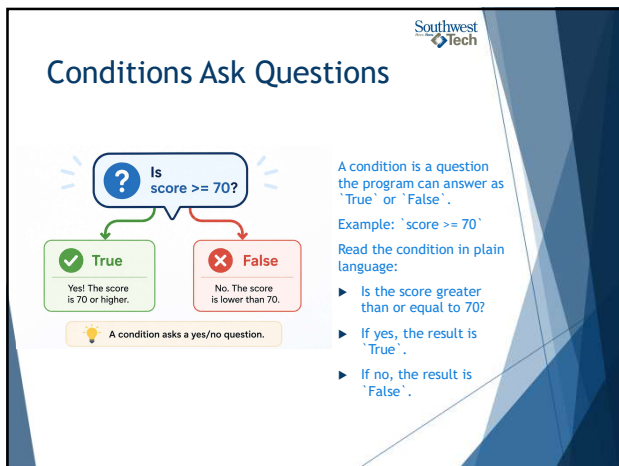
9



Southwest Tech

Review Coding Lab

10



Southwest Tech

Conditions Ask Questions

Is score ≥ 70 ?

True
Yes! The score is 70 or higher.

False
No. The score is lower than 70.

A condition asks a yes/no question.

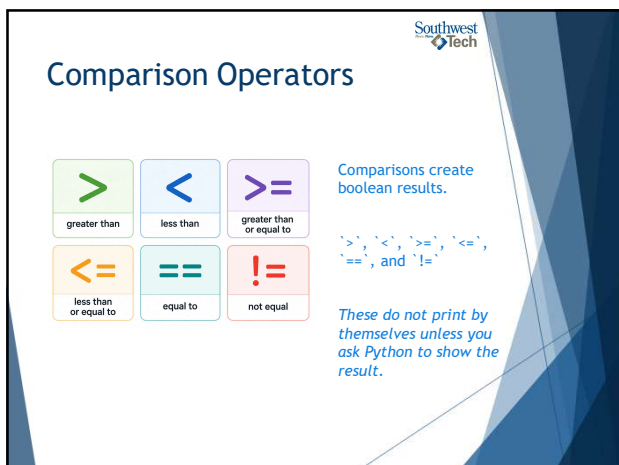
A condition is a question the program can answer as 'True' or 'False'.

Example: `'score >= 70'`

Read the condition in plain language:

- Is the score greater than or equal to 70?
- If yes, the result is 'True'.
- If no, the result is 'False'.

11



Southwest Tech

Comparison Operators

$>$ greater than	$<$ less than	$\geq$ greater than or equal to
$\leq$ less than or equal to	$==$ equal to	$!=$ not equal

Comparisons create boolean results.

`'>', '<', '>=', '<=', '==', and '!='`

These do not print by themselves unless you ask Python to show the result.

12



13

Southwest
Tech

Boolean Comparisons

Before running each comparison, predict the result.
Will Python answer `'True'` or `'False'`?

For each line, identify:

- ▶ the value being checked
- ▶ the comparison operator
- ▶ the expected boolean result

Then run the program to confirm your prediction.

14

Southwest
Tech

The `'if'` Statement

`'if'` means: run this block only when the condition is `'True'`.

The condition decides whether the indented code runs.
Basic shape:

```
if condition:  
    do_this()
```

The indented line belongs to the branch.

15

Southwest Tech

The `if` Statement

Run the indented block only if the condition is `True`.

```
if score >= 70:
    status = "Pass"
    print("Great job!")
```

Condition Is this True?

This block runs if the condition is True.

Indented block: runs only when the condition is True.

Indentation Block of the "If"

16

Southwest Tech

``elif`` and ``else``

``elif`` checks another condition.
``else`` handles what is left when earlier conditions did not run.

Use this structure when there are multiple possible outcomes:

- ▶ ``if`` starts the decision
- ▶ ``elif`` adds another possible condition
- ▶ ``else`` catches the remaining case

17

Demo #2

"Grade Checker"

18



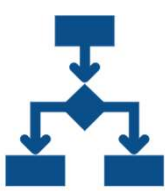
What to watch for

Change the score.
Predict the grade branch before running the program.

For each score, explain:

- ▶ which condition is checked first
- ▶ which condition becomes true
- ▶ which grade is printed
- ▶ why later branches do or do not run

19



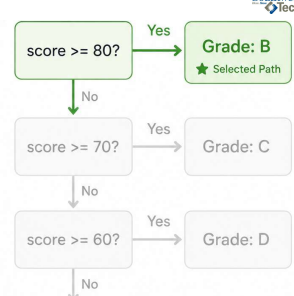
Branch Order Matters

Python checks a branch chain from top to bottom.
The first matching branch wins.

Order matters when more than one condition could be true.

Place the most specific or highest-priority checks where they belong in the chain.

20



```

graph TD
    Q1{score >= 80?} -- Yes --> B[Grade: B  
★ Selected Path]
    Q1 -- No --> Q2{score >= 70?}
    Q2 -- Yes --> C[Grade: C]
    Q2 -- No --> Q3{score >= 60?}
    Q3 -- Yes --> D[Grade: D]
    Q3 -- No --> End(( ))
  
```

**Demo 2:
Grade Check Branches**

21

Southwest
Tech

Common Failure: `=` Is Not `==`

- ▶ `=` stores a value.
- ▶ `==` compares two values.

Example:

```
score = 84    # store the value
score == 84   # ask if the value matches
```

Mixing these up changes the meaning of the program.

22

Southwest
Tech

STORE THE VALUE

Put a value into a variable.

score = 84

84 is stored in score

The value is kept for later use.

COMPARE THE VALUE

Check if a value is equal to another.

score == 84

Are they equal?

The values are checked right now.

Common Failure: Equal sign

23

Southwest
Tech

Common Failure: Indentation Changes Meaning

Indented lines belong to the branch.
Unindented lines run after the branch is finished.

In Python, indentation is structure. (aka: "scope")
When reading a branch, ask:

- ▶ Which lines are inside the branch?
- ▶ Which lines run no matter what?

24

Southwest
Tech

IF STATEMENT STRUCTURE

Indented lines belong to the if branch.

```

1 if score >= 70:
2     grade = "Pass"
3     message = "Great job!"
4 print(grade)
5

```

Inside the branch
These lines run
only if the condition
is True.

After the branch
This line runs
no matter what.

Key Idea:
Indentation groups lines together. Unindented lines are outside the group.

Indentation is about "scope" (in Python)

25

Southwest
Tech

From Demo to Lab

In Lab For Assignment 2:

1. Build one small program that makes a clear decision.
2. Start with two or three test values before adding more features.
3. A good first version should include:
 - ▶ one main value to check
 - ▶ at least one comparison
 - ▶ at least two possible outputs
 - ▶ readable variable names

26

Southwest
Tech

Assignment Options: Decision Programs

1. Grade Checker

Input: score → Condition: score >= 70? → Yes: Output: "Pass" / No: Output: "Fail"

Decide if a score is passing or not.

2. Eligibility Checker

Input: age → Condition: age >= 18? → Yes: Output: "Eligible" / No: Output: "Not eligible"

Decide if a person meets the age requirement.

3. Weather Message

Input: temperature → Condition: temp >= 75? → Yes: Output: "It's warm!" / No: Output: "It's cool!"

Show a message based on the temperature.

4. Simple Recommendation

Input: hours studied → Condition: hours >= 5? → Yes: Output: "Keep it up!" / No: Output: "Study more."

Give a simple tip based on study time.

Assignment 2

27

Lab/Assignment Evidence

Your submission should show at least two possible outcomes.

Be ready to explain what condition is checked and why each branch runs.

Useful evidence includes:

- a working `.py` file`
- sample values or scenarios
- visible output
- a short explanation of the decision

28

Southwest Tech

Code File

```
grade_checker.py
1 score = int(input("Enter score: "))
2 if score >= 70:
3     print("Result: Pass")
4 else:
5     print("Result: Fail")
6
```

Test Values

Test 1	Test 2
Input (score) 85	Input (score) 62
Condition Checked 85 >= 70	Condition Checked 62 >= 70

Output (What the program prints)

Run with 85	Run with 62
Enter score: 85 Result: Pass	Enter score: 62 Result: Fail

Explanation

The program asks for a score. It checks: `score >= 70`. If True, it prints "Pass". If False, it prints "Fail".

★ Evidence: Code file + test values + output + explanation ✓

Evidence Example

29

Optional Demo #3

"Discount Checker"

30



What to watch for

A practical decision program still follows the same pattern:

check a condition -> choose a branch -> show the result.

In the demo, watch how the program:

- ▶ checks the purchase total
- ▶ decides whether a discount applies
- ▶ calculates the final total only when needed
- ▶ prints a result that matches the decision

31



Closing Success Check

If you can predict the branch and explain the output, you understand the logic.

Run the code to confirm your reasoning, not to replace it.

Before submitting Assignment 2, test your program with:

- ▶ one value that should trigger the first branch
- ▶ one value that should trigger another branch
- ▶ one boundary value if your decision has a cutoff

32



Thank you!

Have Fun!

33
